

Hydraulic System Condition Monitoring

MLOps Pipeline — From Training to Production

Lina Rhiati Hazime Mohamed Horache Benjamin Lepourtois Grégoire Petit



DATA713 — March 2026

Context & Problem

Use case: Industrial predictive maintenance

Dataset: UCI — Condition Monitoring of Hydraulic Systems

- 2205 operating cycles $\times$ 17 sensors
- 4 component conditions to predict simultaneously
- Labels from `profile.txt` (supervised)

Formulation: Multi-class multi-output classification

- Why not anomaly detection? Labeled data available
- per-component diagnostics are more actionable

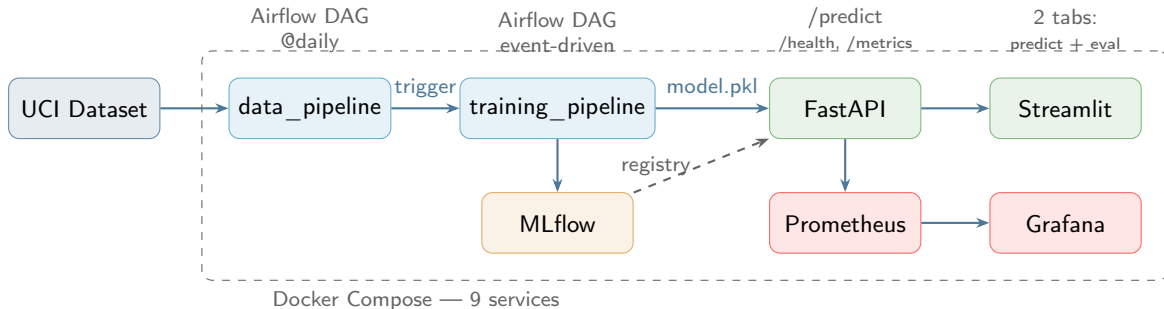
Target	Component	Classes
cooler	Cooler	3
valve	Valve	4
pump	Pump	3
accumulator	Accum.	4

17 sensors:

PS1–PS6, EPS1, FS1–FS2,
TS1–TS4, VS1, CE, CP, SE

Feature eng.: mean per cycle

Architecture Overview



Why Docker Compose?

Single docker compose up → 9 services.
Reproducible across machines, no manual setup.

Why K8s manifests?

Production-ready CD pipeline. Deploy conditional on KUBECONFIG secret (honest: no cluster).

ML Pipeline & Continuous Training

Model: MultiOutputClassifier(RF)

Why? Single training run, consistent features, simpler deployment than 4 models

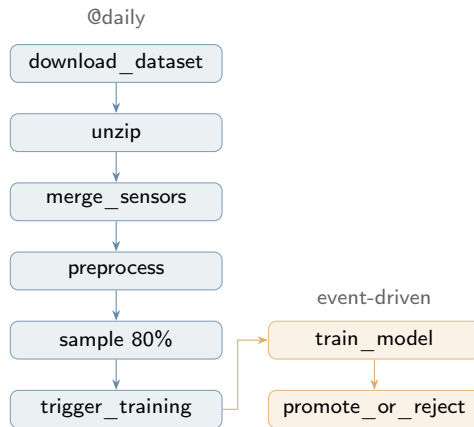
Results (test set 20%):

Target	Acc.	F1 _{macro}	F1 _{w.}
Cooler	1.000	1.000	1.000
Valve	0.948	0.947	0.948
Pump	0.993	0.993	0.993
Accumulator	0.990	0.990	0.990

Design decisions:

- F1 macro for comparison (handles class imbalance)
- DAG delegates to src/train.py (separation of concerns)
- Promote only if $F1_{new} > F1_{prod}$

CT Logic (2 DAGs):



Why 80% sampling on static data?

Simulates data variability → champion/challenger

CI/CD Pipeline

CI (every push & PR)

✓ ruff check

✓ pytest (26 tests)

26 tests in 3 suites:

Suite	#	Tests
DAGs	9	structure, task IDs, deps
Model	7	training, F1, features
Preproc.	10	merge, filter, NaN

CD (push to main)

Run tests

Build Docker images

Push GHCR + DockerHub

check-deploy (secrets?)

▶▶ Deploy K8s (skipped)

Why dual registry? GHCR + DockerHub for redundancy

Tags: latest + git SHA → enables rollback

Package mgr: uv (fast, lockfile, reproducible)

FastAPI

- `POST /predict`
Pydantic validation
17 sensors → 4 states
- `GET /health`
K8s readiness probe
- `GET /metrics`
Prometheus scraping
- `GET /docs`
Swagger auto-gen.

Streamlit

Tab 1 — Prediction

- 17 sensor input fields
- Calls API → 4 states

Tab 2 — Evaluation

- Per-target metrics
- Confusion matrices
- Classification reports

From `model_metrics.json`

→ See screenshots in backup slides

Monitoring

- **Prometheus**
Scrapes `/metrics`
+ Node Exporter
- **Grafana**
Auto-provisioned
on compose up
- **Alerting**
Airflow email callbacks
on DAG failure

Team organization:

Who	Scope
Lina	Data & ML pipeline, MLflow
Grégoire	Airflow, CT, CI/CD, tests, docs
Benjamin	FastAPI API, Streamlit webapp
Mohamed	Docker, K8s, CD, monitoring

Git workflow:

- Feature branches → PR → main
- CI gates on every PR (ruff + pytest)
- 20+ PRs, conventional commits
- Copilot code reviews

Honest limitations:

- Static dataset — CT designed for production but validated on sampled data
- No K8s cluster — CD ready, deploy skipped
- API loads `model.pkl` locally
(prod: would query MLflow Registry)
- Heavy DAG imports → deferred inside task functions (30s parsing limit)

Key design principle:

Separation of concerns:

`src/` = business logic, `dags/` = orchestration,
`api/` = serving, `webapp/` = UI

Objectives Coverage

10/10 required objectives:

#	Objective	Status
1	Data extraction & preprocessing	✓
2	ML model	✓
3	Model registry (MLflow)	✓
4	Airflow retraining pipeline	✓
5	MLflow experiment tracking	✓
6	API (FastAPI + Swagger)	✓
7	WebApp (Streamlit, 2 tabs)	✓
8	Continuous Training (CT)	✓
9	Docker + K8s + CI/CD	✓
10	GitHub versioning & docs	✓

Bonus realized (3/8):

#	Bonus	Status
12	Monitoring (Prom+Grafana)	✓
14	Model versioning/rollback	✓
17	Email alerting	✓
11	Live data	✗
13	Load testing (Locust)	✗
15	Human in the loop	✗
16	API access mgmt	✗

Stack:

Python 3.11 · scikit-learn · MLflow
Airflow · FastAPI · Streamlit
Docker · K8s · GitHub Actions
Prometheus · Grafana · uv



SCREENSHOT

Airflow DAG list
(data_pipeline + training_pipeline)

DAG list

view



SCREENSHOT

data_pipeline task graph
(6 tasks chained)

Task graph view



SCREENSHOT

MLflow experiment runs
(F1 metrics per target)

tracking

Experiment



SCREENSHOT

MLflow Model Registry
(Production / Archived stages)

Model versioning

 SCREENSHOT

Swagger /docs
(Pydantic schema)

FastAPI

 SCREENSHOT

Streamlit Tab 1
(prediction form)

Prediction

 SCREENSHOT

Streamlit Tab 2
(confusion matrices)

Evaluation

Backup — Monitoring & GitHub



SCREENSHOT

Grafana dashboard
(API metrics + system)

Monitoring



SCREENSHOT

GitHub PRs
(20+ merged, CI gates)

Collaboration



SCREENSHOT

GitHub Actions CI
(ruff + pytest green)



SCREENSHOT

CD pipeline
(build + push + check-deploy)